

几何盛宴，震撼美味！

2026.5.2 4:03 cc补题单剩下的感觉全是不可做题啊.....不兑！怎么还有十几道几何，到底是谁塞进去的
来场几何盛宴，以下是2024年至今XCPC的所有几何题

签到蛆

2024ICPC昆明H: [Horizon Scanning - Problem - QOJ.ac](#) ✓

铜牌蛆

2025EC-FINAL杭州A: [Outstanding Outlines - Problem - QOJ.ac](#)

银牌区

2024ICPC沈阳G: [Guess the Polygon - Problem - QOJ.ac](#) ✓

2024CCPC哈尔滨B: [Concave Hull - Problem - QOJ.ac](#) ✓

2025ICPC南京B: [What, More Kangaroos? - Problem - QOJ.ac](#)

2025ICPC沈阳G: [Collision Damage - Problem - QOJ.ac](#) ✓

金牌区

2024ICPC香港E: [Concave Hull - Problem - QOJ.ac](#) ✓

2024ICPC南京M: [Ordainer of Inexorable Judgment - Problem - QOJ.ac](#) ✓

2025ICPC成都I: [Inside Triangle - Problem - QOJ.ac](#) ✓

2025ICPC武汉G: [Projection - Problem - QOJ.ac](#)

2025CCPC重庆C: [GPA Optimization Plan - Problem - QOJ.ac](#)

捧杯区+无人区

2024ICPC成都M: [Two Convex Holes - Problem - QOJ.ac](#)

2024ICPC杭州C: [Catch the Star - Problem - QOJ.ac](#)

2024ICPC上海A: [Ancient Maps, Hidden Danger - Problem - QOJ.ac](#)

2024ICPC上海M: [Machine Learning with Penguins - Problem - QOJ.ac](#)

2024ICPC沈阳K: [Fragile Pinball - Problem - QOJ.ac](#)

2024EC-FINAL西安D: [Keystone Correction - Problem - QOJ.ac](#)

2024CCPC济南G: [The Wheel of Fortune - Problem - QOJ.ac](#)

2025ICPC成都E: [Escaping from Trap - Problem - QOJ.ac](#)

2025ICPC南京L: [Regional Champion - Problem - QOJ.ac](#)

2025ICPC上海F: [Flower's land 4 - Problem - QOJ.ac](#)

2025CCPC济南B: [Fortress - Problem - QOJ.ac](#)

Horizon Scanning

来源：2024ICPC昆明H 难度：签到

题意：给定平面上的 n 个点，要求在原点放置一个雷达，使得雷达在旋转至任意角度时，都能照到至少 k 个点。最小化雷达照射的角度范围

数据范围： $1 \leq T \leq 10^4$, $1 \leq k \leq n \leq 2 \times 10^5$, $|x_i|, |y_i| \leq 10^9$, $\sum n \leq 2 \times 10^5$

题解：使用 `atan2` 等方法，将点转化为极角。此时问题转变为，求一个循环的极角序列上，相隔 k 个位置的极角差值的最大值

```
1  using db = long double;
2  const db pi = acos(-1);
3  const db eps = 1e-8;
4
5  int dcmp(db x, db y) {
6      if(fabs(x - y) < eps) return 0;
7      return x > y ? 1 : -1;
8  }
9
10 void solve() {
11     int n, k;
12     cin >> n >> k;
13     vector<db> a(n);
14     for(int i = 0; i < n; i++) {
15         int x, y;
16         cin >> x >> y;
17         a[i] = atan2(x, y);
18     }
19     sort(a.begin(), a.end());
20     for(int i = 0; i < n; i++)
21         a.push_back(a[i] + pi * 2.0);
22
23     db res = 0;
24     for(int i = 0; i + k < n * 2; i++)
```

```

25         if(dcmp(a[i + k] - a[i], res) == 1)
26             res = a[i + k] - a[i];
27     cout << fixed << setprecision(12) << res << '\n';
28 }

```

Guess the Polygon

来源：2024ICPC沈阳G 难度：银牌

题意：乱序给出一个简单多边形的 n 个顶点，每次可以询问竖直线 $x = p/q$ 与简单多边形的相交部分总长，交互器以分数形式 r/s 返回结果，要在 $n - 2$ 次询问内求出简单多边形的面积，以分数形式 u/v 输出

数据范围： $1 \leq T \leq 1000$ ， $3 \leq n \leq 1000$ ， $0 \leq x, y \leq 1000$ ， $\sum n \leq 1000$

题解：记 $f(x)$ 表示 x 处的竖直线与简单多边形的相交部分总长， $f(x)$ 是分段线性函数，分段点是简单多边形各顶点的横坐标。当 n 个顶点横坐标互不相同时， $f(x)$ 是 $n - 1$ 段连续分段函数，且两端的 f 值为 0 ，只需要询问中间 $n - 2$ 个分段点处的函数值；否则 $f(x)$ 是至多 $n - 2$ 段连续分段函数，询问每一段中点处的函数值

由于交互器返回的 r 和 s 可能都很大，需要特殊处理：观察到简单多边形面积的两倍是不超过 2×10^6 的整数，因此可以任取一个大于 2×10^6 的质数 p ，分数对 p 取模进行计算

```

1  const int mod = 1e9 + 7;
2  typedef long long ll;
3
4  ll qmi(ll m, ll k) {
5      ll res = 1;
6      while(k) {
7          if(k & 1) res = res * m % mod;
8          m = m * m % mod;
9          k >>= 1;
10     }
11     return res;
12 }
13
14 ll inv(ll x) {
15     return qmi(x, mod - 2);
16 }
17
18 ll gcd(ll a, ll b) {
19     return b ? gcd(b, a % b) : a;
20 }
21
22 ll query(ll x, ll y) {

```

```

23     ll g = gcd(x, y);
24     x /= g, y /= g;
25     cout << "? " << x << ' ' << y << endl;
26     string a, b;
27     cin >> a >> b;
28     x = 0, y = 0;
29     for(auto c: a) x = (x * 10 + (c - '0')) % mod;
30     for(auto c: b) y = (y * 10 + (c - '0')) % mod;
31     return x * inv(y) % mod;
32 }
33
34 void solve() {
35     int n;
36     cin >> n;
37     vector<int> x;
38     for(int i = 0; i < n; i++) {
39         int a, b;
40         cin >> a >> b;
41         x.push_back(a);
42     }
43     sort(x.begin(), x.end());
44     x.erase(unique(x.begin(), x.end()), x.end());
45
46     int sz = x.size();
47     ll ans = 0;
48     if(sz == n) {
49         vector<ll> f;
50         f.push_back(0);
51         for(int i = 1; i + 1 < n; i++)
52             f.push_back(query(x[i], 1));
53         f.push_back(0);
54         for(int i = 0; i + 1 < n; i++)
55             ans = (ans + (f[i] + f[i + 1]) * (x[i + 1] - x[i])) % mod;
56     } else {
57         for(int i = 0; i + 1 < sz; i++) {
58             ll v = query(x[i] + x[i + 1], 2);
59             ans = (ans + v * (x[i + 1] - x[i])) % mod;
60         }
61         ans = ans * 211 % mod;
62     }
63
64     if(ans % 2 == 0) cout << "! " << ans / 2 << ' ' << 1 << endl;
65     else cout << "! " << ans << ' ' << 2 << endl;
66 }

```

Concave Hull

来源：2024CCPC哈尔滨B 难度：银牌

题意：给定二维平面上的点集，要求选择若干个点以及点之间的连接顺序，使得这些点连成一个面积严格 > 0 的简单凹多边形，最大化这个凹多边形的面积

数据范围： $1 \leq T \leq 10^4$, $3 \leq n \leq 10^5$, $-10^9 \leq x, y \leq 10^9$, $\sum n \leq 2 \cdot 10^5$, 保证不存在三点共线

题解：首先注意到，凸包上的所有点一定会选上，然后在不在凸包上的点中选择一个凹进去，而这个选择的点，一定是内部凸包上的点。首先找到整个点集的凸包，如果所有点都在凸包上则无解（如果没有保证三点不共线，这个位置要求非严格凸包，将凸包边上的所有点都选进凸包里）。对于内部的点再求一次凸包，对内部凸包上的每个点，找到与外凸包一条边组成的三角形面积最小，可以用双指针实现。最终答案为外凸包面积减去最小的三角形面积

时间复杂度： $O(n \log n)$

标签：凸包

```
1  using ll = long long;
2  using db = long double;
3
4  // =====
5  // 1. 点与向量模版 (Point & Vector)
6  // =====
7  struct Point {
8      ll x, y;
9      int id;
10     Point(ll _x = 0, ll _y = 0, int _id = -1) : x(_x), y(_y), id(_id) {}
11
12     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y, id); }
13     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y, id); }
14
15     bool operator<(const Point& p) const {
16         if (x != p.x) return x < p.x;
17         return y < p.y;
18     }
19     bool operator==(const Point& p) const {
20         return x == p.x && y == p.y;
21     }
22 };
23
24 // =====
25 // 2. 核心几何函数
26 // =====
```

```

27 ll cross(const Point& a, const Point& b) {
28     return a.x * b.y - a.y * b.x;
29 }
30
31 ll cross(const Point& o, const Point& a, const Point& b) {
32     return cross(a - o, b - o);
33 }
34
35 ll dot(const Point& a, const Point& b) {
36     return a.x * b.x + a.y * b.y;
37 }
38
39 ll dot(const Point& o, const Point& a, const Point& b) {
40     return dot(a - o, b - o);
41 }
42
43 int to_left(const Point& a, const Point& b, const Point& c) {
44     ll res = cross(a, b, c);
45     if (res > 0) return 1;
46     if (res < 0) return -1;
47     return 0;
48 }
49
50 bool is_on_segment(const Point& a, const Point& b, const Point& c) {
51     return to_left(a, b, c) == 0 && dot(c, a, b) <= 0;
52 }
53
54 // =====
55 // 3. 多边形模版 (Polygon)
56 // =====
57 struct Polygon {
58     vector<Point> p;
59
60     Polygon() = default;
61     Polygon(const vector<Point>& pts) : p(pts) {}
62
63     void add_point(ll x, ll y, int id = -1) {
64         p.emplace_back(x, y, id);
65     }
66     void add_point(Point x) {
67         p.emplace_back(x);
68     }
69
70     // 计算多边形有向面积的两倍
71     ll double_area() const {
72         ll res = 0;
73         int n = p.size();

```

```

74         for(int i = 0; i < n; i++) {
75             res += cross(p[i], p[(i + 1) % n]);
76         }
77         return abs(res);
78     }
79
80     // Andrew 算法求严格凸包
81     void get_convex_hull() {
82         if(p.size() <= 2) return;
83
84         sort(p.begin(), p.end());
85         vector<Point> hull;
86         int n = p.size();
87
88         for(int i = 0; i < n; i++) {
89             while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
90                 hull.pop_back();
91             }
92             hull.push_back(p[i]);
93         }
94
95         int t = hull.size();
96         for(int i = n - 2; i >= 0; i--) {
97             while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
98                 hull.pop_back();
99             }
100             hull.push_back(p[i]);
101         }
102
103         hull.pop_back();
104         p = hull;
105     }
106 };
107
108 void solve() {
109     int n;
110     cin >> n;
111
112     vector<Point> p(n);
113     Polygon H1;
114     for (int i = 0; i < n; i++) {
115         cin >> p[i].x >> p[i].y;
116         p[i].id = i;
117         H1.add_point(p[i]);
118     }

```

```

119     H1.get_convex_hull();
120     vector<int> st(n, 0);
121     for (auto pt : H1.p)
122         st[pt.id] = 1;
123
124     Polygon H2;
125     for (int i = 0; i < n; i++)
126         if (!st[i])
127             H2.add_point(p[i]);
128     if (H2.p.empty()) {
129         cout << -1 << '\n';
130         return;
131     }
132     H2.get_convex_hull();
133
134     int k = H1.p.size(), m = H2.p.size();
135     ll mn = 0x3f3f3f3f3f3f3f3f;
136     int best_j = 0;
137     for (int j = 1; j < m; j++)
138         if (cross(H1.p[0], H1.p[1], H2.p[j]) < cross(H1.p[0], H1.p[1],
H2.p[best_j]))
139             best_j = j;
140
141     for (int i = 0, j = best_j; i < k; i++) {
142         Point A = H1.p[i];
143         Point B = H1.p[(i + 1) % k];
144         int la = j;
145         do {
146             ll cur_area = cross(A, B, H2.p[j]);
147             ll nxt_area = cross(A, B, H2.p[(j + 1) % m]);
148             if (nxt_area <= cur_area)
149                 j = (j + 1) % m;
150             else break;
151         } while(j != la);
152         mn = min(mn, cross(A, B, H2.p[j]));
153     }
154
155     ll ans = H1.double_area() - mn;
156     cout << ans << '\n';
157 }

```


Collision Damage

来源：2025ICPC沈阳G 难度：银牌

题意：给定凸多边形 P 和 Q ，在 $area(P \cap (Q + t)) > 0$ 的前提下均匀选择 t ，求 $area(P \cap (Q + t))$ 的期望

数据范围： $1 \leq T \leq 300$ ， $3 \leq n, m \leq 1000$ ， $-10^4 \leq x, y \leq 10^4$ ， $\sum n, \sum m \leq 1000$

题解：本质是求， P 和 Q 有交的前提下， Q 均匀随机出现在平面上， P 与 Q 交集面积的期望。交集总面积（即分子）为 P 和 Q 面积的乘积，分母为 P 和 Q 有交的单位时间（用时间这个概念可能有点抽象，反正就是那个意思），可以发现是 P 和 $-Q$ 的闵可夫斯基和（ Q 接触着 P 尽可能大的绕一圈），可以将 P 和 Q 的顶点两两做差后对所有点求凸包面积

时间复杂度：闵可夫斯基和： $O(n + m)$ ，凸包 $O(nm \log(nm))$

标签：凸包，闵可夫斯基和

```
1 using ll = long long;
2 using db = long double;
3
4 // =====
5 // 1. 点与向量模版 (Point & Vector)
6 // =====
7 struct Point {
8     ll x, y;
9     Point(ll _x = 0, ll _y = 0) : x(_x), y(_y) {}
10
11     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
12     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
13
14     bool operator<(const Point& p) const {
15         if (x != p.x) return x < p.x;
16         return y < p.y;
17     }
18     bool operator==(const Point& p) const {
19         return x == p.x && y == p.y;
20     }
21 };
22
23 // =====
24 // 2. 核心几何函数
25 // =====
26
27 // 向量外积 (叉乘)
28 ll cross(const Point& a, const Point& b) {
29     return a.x * b.y - a.y * b.x;
30 }
```

```

31
32 // 三点外积
33 ll cross(const Point& o, const Point& a, const Point& b) {
34     return cross(a - o, b - o);
35 }
36
37 // 向量内积 (点乘)
38 ll dot(const Point& a, const Point& b) {
39     return a.x * b.x + a.y * b.y;
40 }
41
42 // 三点内积
43 ll dot(const Point& o, const Point& a, const Point& b) {
44     return dot(a - o, b - o);
45 }
46
47 /**
48  * To-Left 测试
49  * 返回值:
50  * 1: 点 c 在有向向量 ab 的左侧 (逆时针)
51  * -1: 点 c 在有向向量 ab 的右侧 (顺时针)
52  * 0: 点 c 在直线 ab 上 (共线)
53  */
54 int to_left(const Point& a, const Point& b, const Point& c) {
55     ll res = cross(a, b, c);
56     if (res > 0) return 1;
57     if (res < 0) return -1;
58     return 0;
59 }
60
61 // 判断点 c 是否有向线段 ab 上 (闭区间)
62 bool is_on_segment(const Point& a, const Point& b, const Point& c) {
63     return to_left(a, b, c) == 0 && dot(c, a, b) <= 0;
64 }
65
66 // =====
67 // 3. 多边形模版 (Polygon)
68 // =====
69 struct Polygon {
70     vector<Point> p;
71
72     Polygon() = default;
73     Polygon(const vector<Point>& pts) : p(pts) {}
74
75     void add_point(ll x, ll y) {
76         p.emplace_back(x, y);
77     }

```

```

78
79 // 计算多边形有向面积的两倍
80 ll double_area() const {
81     ll res = 0;
82     int n = p.size();
83     for(int i = 0; i < n; i++) {
84         res += cross(p[i], p[(i + 1) % n]);
85     }
86     return abs(res);
87 }
88
89 // Andrew 算法求凸包
90 void get_convex_hull() {
91     if(p.size() <= 2) return;
92
93     sort(p.begin(), p.end());
94     vector<Point> hull;
95     int n = p.size();
96
97     for(int i = 0; i < n; i++) {
98         while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
99             hull.pop_back();
100         }
101         hull.push_back(p[i]);
102     }
103
104     int t = hull.size();
105     for(int i = n - 2; i >= 0; i--) {
106         while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
107             hull.pop_back();
108         }
109         hull.push_back(p[i]);
110     }
111
112     hull.pop_back();
113     p = hull;
114 }
115 };
116
117 void solve() {
118     int n, m;
119     cin >> n >> m;
120     Polygon S, T;
121
122     for(int i = 0; i < n; i++) {

```

```

123         ll x, y;
124         cin >> x >> y;
125         S.add_point(x, y);
126     }
127     for(int i = 0; i < m; i++) {
128         ll x, y;
129         cin >> x >> y;
130         T.add_point(x, y);
131     }
132
133     Polygon V;
134     for(int i = 0; i < n; i++) {
135         for(int j = 0; j < m; j++) {
136             Point shifted = T.p[j] - S.p[i];
137             V.add_point(shifted.x, shifted.y);
138         }
139     }
140     V.get_convex_hull();
141
142     db s1 = (db)S.double_area() / 2.0;
143     db s2 = (db)T.double_area() / 2.0;
144     db s3 = (db)V.double_area() / 2.0;
145     db res = (s1 * s2) / s3;
146     cout << fixed << setprecision(12) << res << '\n';
147 }

```

```

1  using ll = long long;
2  using db = long double;
3
4  // =====
5  // 1. 点与向量模版 (Point & Vector)
6  // =====
7  struct Point {
8      ll x, y;
9      Point(ll _x = 0, ll _y = 0) : x(_x), y(_y) {}
10
11      Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
12      Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
13
14      bool operator<(const Point& p) const {
15          if (x != p.x) return x < p.x;
16          return y < p.y;
17      }
18      bool operator==(const Point& p) const {
19          return x == p.x && y == p.y;
20      }
21 };

```

```

22
23 // =====
24 // 2. 核心几何函数
25 // =====
26 ll cross(const Point& a, const Point& b) {
27     return a.x * b.y - a.y * b.x;
28 }
29
30 ll cross(const Point& o, const Point& a, const Point& b) {
31     return cross(a - o, b - o);
32 }
33
34 ll dot(const Point& a, const Point& b) {
35     return a.x * b.x + a.y * b.y;
36 }
37
38 ll dot(const Point& o, const Point& a, const Point& b) {
39     return dot(a - o, b - o);
40 }
41
42 int to_left(const Point& a, const Point& b, const Point& c) {
43     ll res = cross(a, b, c);
44     if (res > 0) return 1;
45     if (res < 0) return -1;
46     return 0;
47 }
48
49 bool is_on_segment(const Point& a, const Point& b, const Point& c) {
50     return to_left(a, b, c) == 0 && dot(c, a, b) <= 0;
51 }
52
53 // =====
54 // 3. 多边形模版 (Polygon)
55 // =====
56 struct Polygon {
57     vector<Point> p;
58
59     Polygon() = default;
60     Polygon(const vector<Point>& pts) : p(pts) {}
61
62     void add_point(ll x, ll y) {
63         p.emplace_back(x, y);
64     }
65
66     // 重新排序：使最左下角的点成为起点，保证边向量从第一象限开始
67     void reorder() {
68         if(p.empty()) return;

```

```

69         int id = 0, n = p.size();
70         for(int i = 1; i < n; i++) {
71             if(p[i].y < p[id].y || (p[i].y == p[id].y && p[i].x < p[id].x)) {
72                 id = i;
73             }
74         }
75         rotate(p.begin(), p.begin() + id, p.end());
76     }
77
78     ll double_area() const {
79         ll res = 0;
80         int n = p.size();
81         for(int i = 0; i < n; i++) {
82             res += cross(p[i], p[(i + 1) % n]);
83         }
84         return abs(res);
85     }
86
87     void get_convex_hull() {
88         if(p.size() <= 2) return;
89
90         sort(p.begin(), p.end());
91         vector<Point> hull;
92         int n = p.size();
93
94         for(int i = 0; i < n; i++) {
95             while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
96                 hull.pop_back();
97             }
98             hull.push_back(p[i]);
99         }
100
101         int t = hull.size();
102         for(int i = n - 2; i >= 0; i--) {
103             while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
104                 hull.pop_back();
105             }
106             hull.push_back(p[i]);
107         }
108
109         hull.pop_back();
110         p = hull;
111     }
112 };
113

```

```

114 // =====
115 // 4. 求闵可夫斯基和 (O(n+m))
116 // =====
117 Polygon minkowski(Polygon A, Polygon B) {
118     // 1. 找到各自的左下角基准点
119     A.reorder();
120     B.reorder();
121
122     int n = A.p.size(), m = B.p.size();
123     vector<Point> v1(n), v2(m);
124
125     // 2. 求出所有的边向量 (由于是逆时针凸多边形, 极角严格单调递增)
126     for (int i = 0; i < n; i++) v1[i] = A.p[(i + 1) % n] - A.p[i];
127     for (int i = 0; i < m; i++) v2[i] = B.p[(i + 1) % m] - B.p[i];
128
129     Polygon res;
130     // 起点是两个基准点的和向量
131     res.add_point(A.p[0].x + B.p[0].x, A.p[0].y + B.p[0].y);
132
133     // 3. 双指针归并 (按极角大小合并)
134     int i = 0, j = 0;
135     while (i < n && j < m) {
136         Point cur = res.p.back();
137         if (cross(v1[i], v2[j]) >= 0) { // v1[i] 在 v2[j] 的顺时针方向 (极角更小)
138             res.add_point(cur.x + v1[i].x, cur.y + v1[i].y);
139             i++;
140         } else {
141             res.add_point(cur.x + v2[j].x, cur.y + v2[j].y);
142             j++;
143         }
144     }
145
146     // 4. 将剩余的向量补齐
147     while (i < n) {
148         Point cur = res.p.back();
149         res.add_point(cur.x + v1[i].x, cur.y + v1[i].y);
150         i++;
151     }
152     while (j < m) {
153         Point cur = res.p.back();
154         res.add_point(cur.x + v2[j].x, cur.y + v2[j].y);
155         j++;
156     }
157
158     // 弹出最后一个点 (因为会与起点重合)
159     res.p.pop_back();
160     return res;

```

```

161 }
162
163 void solve() {
164     int n, m;
165     cin >> n >> m;
166     Polygon S, T;
167
168     for(int i = 0; i < n; i++) {
169         ll x, y;
170         cin >> x >> y;
171         S.add_point(x, y);
172     }
173     for(int i = 0; i < m; i++) {
174         ll x, y;
175         cin >> x >> y;
176         T.add_point(x, y);
177     }
178
179     // 为了求 T 和 -S 的闵可夫斯基和，先构建 -S 多边形
180     Polygon neg_S;
181     for(auto pt : S.p) {
182         neg_S.add_point(-pt.x, -pt.y);
183     }
184     // 直接在 O(N+M) 时间内求解闵可夫斯基和
185     Polygon V = minkowski(T, neg_S);
186
187     db s1 = (db)S.double_area() / 2.0;
188     db s2 = (db)T.double_area() / 2.0;
189     db s3 = (db)V.double_area() / 2.0;
190     db res = (s1 * s2) / s3;
191     cout << fixed << setprecision(12) << res << '\n';
192 }

```

Collision Damage

来源：2024ICPC香港E 难度：金牌

题意：平面上有 n 个点，计算给定点集的所有凹包面积之和的两倍，答案对 $10^9 + 7$ 取模

凹包定义：对于一个多边形，给定的 n 个点中，每个点要么位于该多边形内部，要么是該多边形的一個頂點。並且這個多邊形恰好有一個內角大於 π ，其它所有內角都小於 π

数据范围： $3 \leq n \leq 2 \cdot 10^3$, $0 \leq x_i, y_i \leq 10^9$

题解：首先，由于 n 个点都要位于凹包内部或边界上，全局凸包上的点，一定也是凹包的顶点。唯一的凹包顶点 p_i 必定是严格位于全局凸包内部的某一个点，合法的凸包，本质上就是从全局凸包的某一条边 $C_m \rightarrow C_{m+1}$ 开始，向内部挖去了一个顶点为 p_i 的“星形口袋”。为了确保原本在凸包内部的其它点，在挖去口袋后依然留在凹包内部，这个口袋的左边界和右边界，必须分别是这些点相对于 p_i 的局部凸壳

对 n 个点先求出全局凸包，随后枚举内部的点 p_i 。将除了 p_i 以外的所有点以 p_i 为极点进行极角排序，凹角 $\angle Ap_i B$ 一定是由极角排序后两个相邻的点构成，如果另外两个点 A 和 B 不是相邻的，那个这个多边形会发生自交。全局凸壳的每一条边 $C_m \rightarrow C_{m+1}$ 在 p_i 的视角下，正好切分出了一个扇形极角区间，利用极角排序后记录的索引，可以提取出被夹在这个扇形内的所有其它内部点。分别从 C_m 和 C_{m+1} 出发，扫描出口袋内缩的左侧轮廓和右侧轮廓，动态维护左右两侧的挖去的面积，即可算出该“星形口袋”挖去的面积

时间复杂度： $O(n^2 \log n)$

标签：凸包，极角排序

```
1 using ll = long long;
2 using db = long double;
3 const ll mod = 1e9 + 7;
4
5 // =====
6 // 1. 点与向量模版 (Point & Vector)
7 // =====
8 struct Point {
9     ll x, y;
10    int id;
11    Point(ll _x = 0, ll _y = 0, int _id = -1) : x(_x), y(_y), id(_id) {}
12
13    Point operator+(const Point& p) const { return Point(x + p.x, y + p.y, id); }
14    Point operator-(const Point& p) const { return Point(x - p.x, y - p.y, id); }
15
16    bool operator<(const Point& p) const {
17        if (x != p.x) return x < p.x;
18        return y < p.y;
19    }
20    bool operator==(const Point& p) const {
21        return x == p.x && y == p.y;
22    }
23 };
24
25 // =====
26 // 2. 核心几何函数
27 // =====
28 ll cross(const Point& a, const Point& b) {
29     return a.x * b.y - a.y * b.x;
```

```

30 }
31
32 ll cross(const Point& o, const Point& a, const Point& b) {
33     return cross(a - o, b - o);
34 }
35
36 ll dis2(const Point& a, const Point& b) {
37     return (a.x - b.x) * (a.x - b.x) + (a.y - b.y) * (a.y - b.y);
38 }
39
40 ll area(const Point& o, const Point& a, const Point& b) {
41     ll res = cross(o, a, b);
42     if(res < 0) res = -res;
43     return res % mod;
44 }
45
46 // 1: c 在 ab 左侧, -1: c 在 ab 右侧, 0: c 在 ab 上
47 int to_left(const Point& a, const Point& b, const Point& c) {
48     ll res = cross(a, b, c);
49     if (res > 0) return 1;
50     if (res < 0) return -1;
51     return 0;
52 }
53
54 // =====
55 // 3. 多边形模版 (Polygon)
56 // =====
57 struct Polygon {
58     vector<Point> p;
59
60     Polygon() = default;
61     Polygon(const vector<Point>& pts) : p(pts) {}
62
63     void add_point(ll x, ll y, int id = -1) {
64         p.emplace_back(x, y, id);
65     }
66     void add_point(Point x) {
67         p.emplace_back(x);
68     }
69
70     // 计算多边形有向面积的两倍
71     ll double_area() const {
72         ll res = 0;
73         int n = p.size();
74         for(int i = 0; i < n; i++) {
75             res += cross(p[i], p[(i + 1) % n]);
76             res = (res % mod + mod) % mod;

```

```

77     }
78     return res;
79 }
80
81 // Andrew 算法求严格凸包
82 void get_convex_hull() {
83     if(p.size() <= 2) return;
84
85     sort(p.begin(), p.end());
86     vector<Point> hull;
87     int n = p.size();
88
89     for(int i = 0; i < n; i++) {
90         while(hull.size() > 1 && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
91             hull.pop_back();
92         }
93         hull.push_back(p[i]);
94     }
95
96     int t = hull.size();
97     for(int i = n - 2; i >= 0; i--) {
98         while(hull.size() > t && to_left(hull[hull.size() - 2],
hull.back(), p[i]) <= 0) {
99             hull.pop_back();
100         }
101         hull.push_back(p[i]);
102     }
103
104     hull.pop_back();
105     p = hull;
106 }
107 };
108
109 // =====
110 // 4. 极角排序
111 // =====
112 // 0: 上半平面, 极角 [0, pi); 1: 下半平面, 极角 [pi, pi*2); -1: 与 base 重合
113 int get_quad(const Point& p, const Point& base) {
114     if(p.x == base.x && p.y == base.y) return -1;
115     if(p.y > base.y || (p.y == base.y && p.x > base.x)) return 0;
116     return 1;
117 }
118
119 void polar_sort(vector<Point>& pts, const Point& base) {
120     sort(pts.begin(), pts.end(), [&](const Point& a, const Point& b) {
121         int qa = get_quad(a, base);

```

```

122         int qb = get_quad(b, base);
123
124         if(qa != qb) return qa < qb;
125
126         ll c = cross(base, a, b);
127         if(c != 0) return c > 0;
128
129         return dis2(base, a) < dis2(base, b);
130     });
131 }
132
133 void solve() {
134     int n;
135     cin >> n;
136     Polygon S;
137     for(int i = 0; i < n; i++) {
138         ll x, y;
139         cin >> x >> y;
140         S.add_point(x, y, i);
141     }
142
143     // 全局凸包
144     Polygon CH = S;
145     CH.get_convex_hull();
146     int ch_sz = CH.p.size();
147
148     if(ch_sz < 3) {
149         cout << "0\n";
150         return;
151     }
152
153     ll ch_area = CH.double_area();
154     vector<int> in_ch(n, 0);
155     for(const Point& pt: CH.p)
156         in_ch[pt.id] = 1;
157
158     ll ans = 0;
159     // 遍历每一个凹点
160     for(const Point& pi: S.p) {
161         if(in_ch[pi.id]) continue;
162
163         vector<Point> pts;
164         for(const Point& pt: S.p)
165             if(pt.id != pi.id)
166                 pts.push_back(pt);
167         polar_sort(pts, pi);
168         vector<int> pos(n);

```

```

169     for(int i = 0; i < n - 1; i++)
170         pos[pts[i].id] = i;
171
172     for(int i = 0; i < ch_sz; i++) {
173         Point S = CH.p[i], T = CH.p[(i + 1) % ch_sz];
174         int idx1 = pos[S.id], idx2 = pos[T.id];
175         vector<Point> Q;
176         Q.push_back(S);
177         int cur = (idx1 + 1) % (n - 1);
178         while(cur != idx2) {
179             Q.push_back(pts[cur]);
180             cur = (cur + 1) % (n - 1);
181         }
182         Q.push_back(T);
183         int sz = Q.size();
184
185         // GRahma 分别进行正向, 反向扫描
186         vector<Point> L, R;
187         ll area_L = 0, area_R = 0;
188         vector<ll> AREA_L(n), AREA_R(n);
189         L.push_back(Q[0]);
190         for(int j = 1; j < sz - 1; j++) {
191             while(L.size() >= 2) {
192                 Point A = L[L.size() - 2], B = L.back();
193                 if(to_left(A, B, Q[j]) <= 0) {
194                     area_L = (area_L - area(pi, A, B) + mod) % mod;
195                     L.pop_back();
196                 } else break;
197             }
198             Point A = L.back();
199             area_L = (area_L + area(pi, A, Q[j])) % mod;
200             AREA_L[j] = area_L;
201             L.push_back(Q[j]);
202         }
203         R.push_back(Q[sz - 1]);
204         for(int j = sz - 2; j >= 1; j--) {
205             while(R.size() >= 2) {
206                 Point A = R[R.size() - 2], B = R.back();
207                 if(to_left(A, B, Q[j]) >= 0) {
208                     area_R = (area_R - area(pi, A, B) + mod) % mod;
209                     R.pop_back();
210                 } else break;
211             }
212             Point A = R.back();
213             area_R = (area_R + area(pi, A, Q[j])) % mod;
214             AREA_R[j] = area_R;
215             R.push_back(Q[j]);

```

```

216         }
217
218         ll area_base = area(pi, S, T);
219         for(int j = 0; j + 1 < sz; j++) {
220             ll s = area_base - AREA_L[j] - AREA_R[j + 1];
221             s = ch_area - s;
222             s = (s % mod + mod) % mod;
223             ans = (ans + s) % mod;
224         }
225     }
226 }
227 cout << ans << '\n';
228 }

```

Ordainer of Inexorable Judgment

来源：2024ICPC南京M 难度：金牌

题意：水神的攻击范围是一条以原点为端点的射线，和所有距离射线 $\leq d$ 的点的集合。射线的初始方向向量给定，会以每个单位时间 1 弧度的顺序旋转。给定一个 n 个顶点的凸多边形，求区间 $[0, t]$ 中攻击范围与目标有交的时间区间长度

数据范围： $3 \leq n \leq 100$ ， $-10^4 \leq x_0, y_0 \leq 10^4$ ， $x_0^2 + y_0^2 > 0$ ， $1 \leq d, t \leq 10^4$ ， $-10^4 \leq x_i, y_i \leq 10^4$ ，保证凸多边形目标距离原点 $> d$

题解：首先计算能攻击到目标的极角范围。能攻击到目标当且仅当存在射线上的一个点距离凸多边形 $\leq d$ 。考虑将凸多边形扩展 d 的大小。具体地，将凸多边形的每个顶点扩展成一个半径为 d 的圆，连接形成一个顶点圆滑的凸多边形。能攻击到目标当且仅当射线与这个新的图形相交

为了求出可行的极角区间，只需要求出极角最小和最大的切线的极角即可。过原点的新图形的切线，一定与某个顶点扩展成的圆相切

求出原点过所有圆的切线。由于区间可能会跨过 2π 的位置，无法直接求最大最小值。但由于题目保证凸多边形距离原点 $> d$ ，也就是必然存在一个半平面满足凸多边形扩展成的图形在这个半平面内。在半平面内取出最靠左和最靠右的切线即可

将时间区间 $[0, t]$ 切分成若干个整周期 $[0, 2\pi]$ 和一个不整的区间 $[0, \theta]$ ，与可行区间求交即可计算答案

时间复杂度： $O(n)$

标签：圆和切线

```

1 using db = long double;
2 const db eps = 1e-8;
3 const db pi = acos(-1);
4 const db inf = 2e36;

```

```

5
6 int dcmp(db x) {
7     if (fabs(x) < eps) return 0;
8     return x < 0 ? -1 : 1;
9 }
10
11 // =====
12 // 1. 点与向量模版 (Point & Vector)
13 // =====
14 struct Point {
15     db x, y;
16     Point(db _x = 0, db _y = 0) : x(_x), y(_y) {}
17
18     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
19     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
20     Point operator*(const db& k) const { return Point(x * k, y * k); }
21     Point operator/(const db& k) const { return Point(x / k, y / k); }
22
23     db length() const { return sqrt(x * x + y * y); }
24
25     // 向量极角 [-pi, pi]
26     db polar_angle() const { return atan2(y, x); }
27
28     // 向量旋转 (逆时针旋转 rad 弧度)
29     Point rotate(db rad) const {
30         return Point(x * cos(rad) - y * sin(rad), x * sin(rad) + y *
cos(rad));
31     }
32 };
33
34 db dist(const Point& a, const Point& b) { return (a - b).length(); }
35
36 // =====
37 // 2. 圆的模版与切点求法 (Circle & Tangents)
38 // =====
39 struct Circle {
40     Point o; // 圆心
41     db r;    // 半径
42     Circle(Point _o = Point(), db _r = 0) : o(_o), r(_r) {}
43 };
44
45
46 // 求点 P 到圆 C 的切点
47 vector<Point> get_tangent_points(const Circle& C, const Point& P) {
48     vector<Point> res;
49     db d = dist(C.o, P);
50     int cmp = dcmp(d - C.r);

```

```

51
52     if (cmp < 0) return res; // 点在圆内
53     if (cmp == 0) { // 点在圆上
54         res.push_back(P);
55         return res;
56     }
57     // 点在圆外
58     Point v = P - C.o;
59     db theta = acos(C.r / d);
60     Point u = v * (C.r / d);
61     res.push_back(C.o + u.rotate(theta));
62     res.push_back(C.o + u.rotate(-theta));
63     return res;
64 }
65
66 // =====
67 // 3. 极角区间与周期处理辅助函数
68 // =====
69 // 规范化角度到 [-pi, pi) 之间
70 db normalize(db angle) {
71     while (angle >= pi) angle -= pi * 2;
72     while (angle < -pi) angle += pi * 2;
73     return angle;
74 }
75
76 // 周期性区间前缀求交
77 db get_intersection_length(db x, db delta) {
78     db K = floor(x / (pi * 2));
79     db rem = x - K * pi * 2;
80     return K * delta + max(0.0L, min(delta, rem));
81 }
82
83 void solve() {
84     int n;
85     db x0, y0, d, t;
86     cin >> n >> x0 >> y0 >> d >> t;
87
88     vector<Point> p(n);
89     for(int i = 0; i < n; i++)
90         cin >> p[i].x >> p[i].y;
91     Point p0(0, 0);
92
93     // 参考极角: 选取第一个顶点的极角
94     db theta_ref = p[0].polar_angle();
95     db mn = inf, mx = -inf;
96     for(int i = 0; i < n; i++) {
97         Circle C(p[i], d);

```



```

98         vector<Point> tangents = get_tangent_points(C, p0);
99
100        for (const Point& T : tangents) {
101            db ang = T.polar_angle();
102            ang = normalize(ang - theta_ref);
103            mn = min(mn, ang);
104            mx = max(mx, ang);
105        }
106    }
107    db L = theta_ref + mn, R = theta_ref + mx;
108    db delta = R - L;
109
110    // 初始朝向 S, 结束朝向 E
111    db S = atan2(y0, x0);
112    db E = S + t;
113    S -= L, E -= L;
114
115    db ans = get_intersection_length(E, delta) - get_intersection_length(S,
116    delta);
117    cout << fixed << setprecision(12) << ans << '\n';
118 }

```

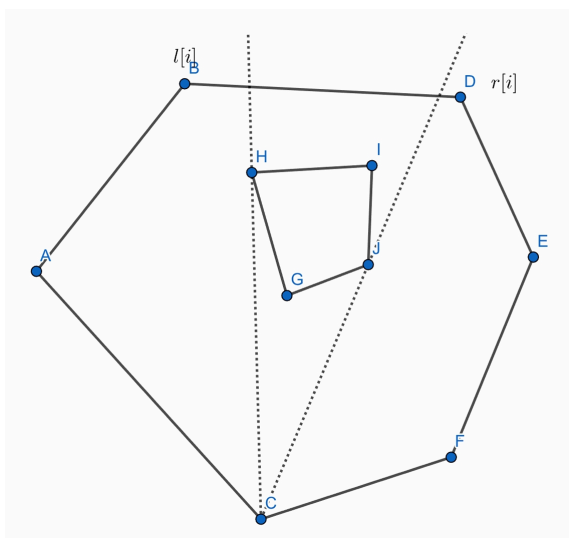
Inside Triangle

来源：2025ICPC成都I 难度：金牌

题意：给定两个凸包，小凸包严格包含于大凸包内。求在大凸包上选三点，三点构成的三角形包含小凸包的方案数

数据范围： $1 \leq T \leq 1000$, $3 \leq n, m \leq 3 \cdot 10^5$, $|x|, |y| \leq 10^9$, $\sum n, \sum m \leq 5 \cdot 10^5$

题解：对大凸包上的每个点 i , 求出其往前往后连边的最大范围 $l[i]$ 和 $r[i]$



当逆时针遍历大凸包上的点 i 时，其在小凸包上的切点也在顺时针旋转，可以通过双指针求出 l 和 r 数组

考虑如何计数，一个 $\triangle UVW$ （逆时针顺序）符合条件等价于小凸包在直线 UV, VW, WU 的左侧。对于每个点 i ，以其为顶点能构成的三角形数量即为区间 $[l[i], i-1]$ 中取一点 V ，区间 $[i+1, r[i]]$ 中取一点 U ，满足小凸包在直线 UV 左侧的方案数。动态维护两个区间的配对数量

考虑逆时针遍历大凸包上的点 i ，动态维护区间 $A = [l[i], i-1]$ 与区间 $B = [i+1, r[i]]$ 中点对的配对数量。从点 i 转移到点 $i+1$ 时，区间变化为 $A: [l[i], i-1] \rightarrow [l[i+1], i]$ ， $B: [i+1, r[i]] \rightarrow [i+2, r[i+1]]$ ，往两个集合中删减点并动态维护配对数量即可

每个三角形被计算了三遍，最后答案要除以三

时间复杂度： $O(n+m)$

```
1  using ll = long long;
2  using db = long double;
3
4  struct Point {
5      ll x, y;
6      Point(ll _x = 0, ll _y = 0) : x(_x), y(_y) {}
7
8      Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
9      Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
10 };
11
12 ll cross(const Point& a, const Point& b) {
13     return a.x * b.y - a.y * b.x;
14 }
15
16 ll cross(const Point& o, const Point& a, const Point& b) {
17     return cross(a - o, b - o);
18 }
19
20 int to_left(const Point& a, const Point& b, const Point& c) {
21     ll res = cross(a, b, c);
22     if (res > 0) return 1;
23     if (res < 0) return -1;
24     return 0;
25 }
26
27 void solve() {
28     int n, m;
29     cin >> n;
30     vector<Point> P(2 * n);
31     for (int i = 0; i < n; i++) {
32         cin >> P[i].x >> P[i].y;
33         P[i + n] = P[i];
```

```

34     }
35     cin >> m;
36     vector<Point> Q(m);
37     for (int i = 0; i < m; i++)
38         cin >> Q[i].x >> Q[i].y;
39
40     vector<int> R(n), L(n);
41     int r_q = 0, l_q = 0;
42     for (int j = 0; j < m; j++) {
43         if (to_left(P[0], Q[r_q], Q[j]) < 0) r_q = j;
44         if (to_left(P[0], Q[l_q], Q[j]) > 0) l_q = j;
45     }
46     R[0] = r_q, L[0] = l_q;
47     for (int i = 1; i < n; i++) {
48         r_q = R[i - 1];
49         while (to_left(P[i], Q[r_q], Q[(r_q + 1) % m]) < 0) r_q = (r_q + 1) %
m;
50         R[i] = r_q;
51
52         l_q = L[i - 1];
53         while (to_left(P[i], Q[l_q], Q[(l_q + 1) % m]) > 0) l_q = (l_q + 1) %
m;
54         L[i] = l_q;
55     }
56
57     vector<int> r(n * 2), l(n * 2);
58     int curr_r = 1, curr_l = 1;
59     for (int i = 0; i < n; i++) {
60         curr_r = max(curr_r, i + 1);
61         while (curr_r < i + n && to_left(P[i], Q[R[i]], P[curr_r]) <= 0)
62             curr_r ++;
63         r[i] = curr_r - 1;
64
65         curr_l = max(curr_l, i + 1);
66         while (curr_l < i + n && to_left(P[i], Q[L[i]], P[curr_l]) < 0)
67             curr_l ++;
68         l[i] = curr_l;
69     }
70     for (int i = 0; i < n; i++) {
71         r[i + n] = r[i] + n;
72         l[i + n] = l[i] + n;
73     }
74
75
76     // pos[x]: 第一个满足 r[u] >= x 的点 u
77     vector<int> pos(n * 3 + 1, n * 2);
78     int p = 0;

```

```

79     for (int x = 0; x <= n * 3; x++) {
80         while (p < n * 2 && r[p] < x) p ++;
81         pos[x] = p;
82     }
83
84     ll S = 0;
85     // A = [l[0], n-1], B = [1, r[0]]
86     int A_L = l[0], A_R = l[0] - 1;
87     int B_L = 1, B_R = 0;
88
89     auto add_A = [&](int v) {
90         S += max(0LL, (ll)min(B_R, v - 1) - max(B_L, pos[v]) + 1);
91     };
92     auto remove_A = [&](int v) {
93         S -= max(0LL, (ll)min(B_R, v - 1) - max(B_L, pos[v]) + 1);
94     };
95     auto add_B = [&](int u) {
96         S += max(0LL, (ll)min(A_R, r[u]) - max(A_L, u + 1) + 1);
97     };
98     auto remove_B = [&](int u) {
99         S -= max(0LL, (ll)min(A_R, r[u]) - max(A_L, u + 1) + 1);
100    };
101
102    while (B_R < r[0]) add_B(B_R ++);
103    while (A_R < n - 1) add_A(++ A_R);
104
105    ll ans = S;
106    for (int i = 0; i < n - 1; i++) {
107        while (A_R < i + n) add_A(++ A_R);
108        while (A_L < l[i + 1]) remove_A(A_L ++);
109        while (B_R < r[i + 1]) add_B(++ B_R);
110        while (B_L < i + 2) remove_B(B_L ++);
111        ans += S;
112    }
113    cout << ans / 3 << '\n';
114 }

```